11a — Monolithic Apps

Key Concepts

- **Monolith** — single deployable unit where all features run in one process
- **Single-process monolith** — everything in one process, most common type
- **Modular monolith** — one deployable but code organized into clear internal modules
- **Distributed monolith** — deployed as separate services but tightly coupled; worst of both worlds
- Modules communicate via **direct function calls** — no network overhead

How It Works

```
User Request
     ↓
Single App Process
  ├── Auth Module
  ├── Order Module
  ├── Payment Module
  └── Notification Module
     ↓
Single Database
```

- All modules share same memory, same process, same database
- One deployable artifact (.exe, .jar, single process)
- One log stream, one stack trace — easy to debug

Strengths vs Weaknesses

Strength	Weakness
Simple to develop early on	Must scale entire app, not individual parts
Easy to test — everything local	Deployment risk — any change redeploys everything
Fast internal calls (function calls)	Team bottleneck — merge conflicts at scale
Simple deployment	Technology lock-in — one language/framework
Easy to debug	One bug can crash entire app

When to Use

- Early-stage startups — ship fast
- Small teams (< 10 engineers)
- Domain not yet well understood — wrong microservice boundaries are costly
- Internal tools with predictable, low load

Industry Examples

- **Amazon (early 2000s)** — started as monolith; deployment fear drove move to microservices → became AWS
- **Stack Overflow** — still largely monolith today, handles millions of requests (monoliths aren't inherently bad)

Interview Questions

Q: What is a monolithic architecture? A: A single deployable unit where all features — auth, payments, notifications — run in one process sharing one database.

Q: What's the main disadvantage of a monolith at scale? A: Can't scale individual components — must scale the whole app. Deployments are also risky because any change touches the entire system.

Q: Is a monolith always bad? A: No. Right choice early on — lower complexity, faster development. Problems emerge when team size and traffic outgrow a single codebase.

Q: What is a distributed monolith? A: Services deployed separately but tightly coupled — shared database, synchronous dependencies — so they can't function independently. Worst of both worlds: distributed complexity with no independence benefit.